

Competitive Programming

Appendix A Editorial: Lazy Propagation

Competitive Programming Course

2026-07-10

Table of contents

1	Exercise 1: range add, range sum	3
1.1	State and operations	3
1.2	Correctness	3
1.3	Complexity	3
2	Exercise 2: range assign, range sum	4
2.1	State and composition	4
2.2	Correctness	4
3	Exercise 3: binary flip, count ones	5
3.1	State and composition	5
3.2	Correctness	5
4	Stress testing	6

1 Exercise 1: range add, range sum

1.1 State and operations

For each node store `sum` and `lazy_add`. Applying `x` to a segment of length `len` sets `sum += x*len` and `lazy_add += x`. Pushing applies the same tag to both children and resets the parent tag to zero.

1.2 Correctness

`apply` changes every value of a fully covered segment by `x`, so its sum rises by exactly `x*len`. Its tag records that children have not yet received that same operation. `push` invokes this correct full-cover operation on each child; their sums and tags become correct before recursion. Merging correct child sums therefore restores the parent invariant. The induction described in the lesson proves both updates and queries.

1.3 Complexity

An update/query visits $O(\log n)$ boundary nodes plus fully covered nodes whose work stops immediately, so it is $O(\log n)$ time and $O(n)$ memory.

2 Exercise 2: range assign, range sum

2.1 State and composition

Store `sum`, `has_assign`, and `assign_value`. Applying assignment `x` sets `sum = x*len`, `has_assign = true`, and `assign_value = x`. A newer assignment replaces the older pending assignment; it does not add to it.

2.2 Correctness

The latest assignment is the only relevant deferred operation because it overwrites every earlier value. Thus replacing the pending tag preserves the meaning “operation still owed to both children.” The same apply/push/pull invariant proves correctness.

3 Exercise 3: binary flip, count ones

3.1 State and composition

Store `ones` and `lazy_flip`. Applying a flip to length `len` changes `ones` to `len-ones` and toggles `lazy_flip`. Two flips cancel, which is why tag composition is xor.

3.2 Correctness

Every zero becomes one and every one becomes zero, so exactly the previous zeros, `len-ones`, are ones afterward. Xor records the parity of unapplied flips; parity completely determines their effect. Push and pull preserve the same node invariant as in Exercise 1.

4 Stress testing

For each variant, generate arrays of length at most eight and random operations. Maintain a plain list beside the tree. After every update, query every interval and compare. Include nested full-range updates because they expose composition order errors that random single-point updates often miss.